降りタイマー (oriTimer) 基本設計書

基本設計書とは ?

基本設計書は ﹁何を作るか﹂ を定義するドキュメントです ゜
詳細設計書 ︿どう作るか﹀ の **前段階** にあたります ゜

基本設計書と詳細設計書の違い:

観点	基本設計書	詳細設計書
読む人	発注者·PM·開発者	開発者
書く内容	何を作るか ︿要件·機能﹀	どう作るか ︿実装方法﹀
技術の深さ	技術に依存しない記述が理想	具体的な型名·関数名まで記載
例	﹁駅に近づいたら通知する﹂	﹁NativeGeofenceManagerでentryイベントを500m

基本設計書は以下の観点で構成されます:

セクション	書く内容	なぜ必要か
1. プロジェクト概要	背景·目的·スコープ	﹁なぜこのアプリを作るのか﹂ を共有す
2. システム構成	アプリとOSサービスの関係	全体の登場人物を明確にするため
3. 機能一覧	アプリが持つ全機能のリスト	﹁何ができて何ができないか﹂ を合意す
4. 機能要件	各機能の振る舞い	開発者の ﹁こう解釈した﹂ を防ぐため
5. 画面一覧·遷移	画面の構成と流れ	ユーザー体験を設計するため
6. データ設計	扱うデータの種類と関係	APIとUIの整合性を保つため
7. 外部インターフェース	API仕様の概要	サーバーとの約束を定義するため
8. 非機能要件	性能·セキュリティ等	﹁動くけど使えない﹂ を防ぐため

1. プロジェクト概要

> **このセクションの目的**：﹁なぜこのアプリを作るのか﹂﹁誰が使うのか﹂﹁どこまでが範囲か﹂を明確にする。プロジェクトに関わる全員が最初に読むセクション。

1.1 背景

電車通勤・通学時に居眠りで目的の駅を乗り越してしまう問題は、時刻指定アラームでは根本的に解決しにくい。電車の遅延や乗り換えが多いルートでは、目的駅の到着時刻を事前に正確に把握することが難しいためである。

本アプリは、スマートフォンのジオフェンス機能を活用し、﹁時刻﹂ではなく﹁場所（駅）﹂をトリガーとした降車通知を実現する。個人学習を兼ねて、Flutter + ネイティブジオフェンスAPIの構成で開発する。

1.2 目的

- 選択した東京の駅の半径1000m圏内に入ったとき、プッシュ通知・ループバイブレーションでユーザーに通知する
- 時刻に依存せず、場所ベースで確実に乗り越しを防ぐ
- Flutter / ジオフェンスAPI / Riverpod の学習教材としても活用する

1.3 対象ユーザー

- 開発者自身（個人利用）
- 電車通勤・通学中に乗り越しが心配な知人（将来的な共有を想定）

1.4 スコープ（対象範囲）

> **補足**：スコープとは﹁このプロジェクトで作るもの・作らないもの﹂の境界線。これを書いておかないと、﹁あれも欲しい、これも欲しい﹂と際限なく膨らんでしまう（スコープクリープと呼ぶ）。

対象範囲（作るもの）:

- 東京の主要路線・駅の一覧表示（APIから取得）
- 降りたい駅の選択機能

- 選択駅のジオフェンス監視の開始·停止
- 駅圏内進入時のプッシュ通知︵バイブレーション付き︶
- 位置情報·通知パーミッションの要求
- 監視状態·選択駅の画面表示

対象外︵作らないもの︶:

- 東京以外の路線·駅への対応
- 複数駅の同時監視
- オフライン対応︵駅データのローカルキャッシュ︶
- 乗車履歴の記録
- 地図表示
- 時刻表の表示

---

2. システム構成

> **このセクションの目的**：アプリがどんな﹁部品﹂で成り立っているかを図示する。﹁アプリとOS·外部APIの関係﹂﹁どこに何があるか﹂を把握するための地図。

2.1 全体構成図

```
┌──────────────────────────────────────────────────────────────┐
│                      モバイル端末（iOS / Android）                │
│                                                                │
│   ┌──────────────────────┐       ┌──────────────────────────┐  │
│   │   Flutter アプリ       │       │  OS ジオフェンスサービス    │  │
│   │                       │       │  (iOSは CoreLocation,     │  │
│   │  - 画面表示・操作       │◄──────►│   AndroidはGeofence API) │  │
│   │  - 駅データ管理        │       │                          │  │
│   │  - 通知発火ロジック     │       │  - 境界線の監視           │  │
│   └──────────┬────────────┘       │  - 入場イベント発火        │  │
│              │                     └──────────────────────────┘  │
│              │ HTTP POST                                          │
└──────────────┼──────────────────────────────────────────────────┘
               │
               ▼
┌──────────────────────┐
│   外部APIサーバー       │
│   (toyohide.work)     │
│                       │
│  - 東京路線・駅データ   │
│    の提供              │
└──────────────────────┘
```

2.2 各コンポーネントの役割

コンポーネント	技術	役割
Flutterアプリ	Flutter (Dart)	ユーザーインターフェース°駅選択·監視開始·通知発火
OSジオフェンスサービス	iOS: CoreLocation / Android: Geofence API	端末の位置情報を監視し`境界線の通過イベントを発火
外部APIサーバー	toyohide.work	東京の路線·駅データをJSON形式で提供

2.3 通信方式

項目	内容
プロトコル	HTTP
データ形式	JSON
通信方向	クライアント → サーバー（リクエスト/レスポンス型）
通信方法	HTTP POST
認証	なし

2.4 ジオフェンスの仕組み

> **補足**：ジオフェンスはネイティブのOS機能を利用するため、Flutterアプリが終了した状態でも動作する。OSがジオフェンス境界の通過を検知し、登録済みのコールバック関数を呼び出す。

```
1. Flutterアプリがジオフェンスを登録（駅の座標 + 半径1000m）
            │
            ▼
2. OSがバックグラウンドで位置情報を監視
            │
            ▼
3. デバイスが駅の1000m圏内に入る
            │
            ▼
4. OSがコールバック関数を呼び出す
            │
            ▼
5. コールバック内でプッシュ通知を発火する
```

---

3. 機能一覧

> **このセクションの目的**：アプリが持つ機能を一覧化する。各機能に番号を振ることで、

後のセクションや議論で﹁F-03の仕様について﹂のように参照できる。

機能ID	機能名	概要
F-01	駅データ取得	起動時に外部APIから東京の全路線・駅データを取得する
F-02	路線・駅一覧表示	取得した駅データを路線ごとにグループ化して表示する
F-03	駅選択	一覧から降りたい駅を1つ選択する
F-04	ジオフェンス監視開始	選択した駅の座標にジオフェンスを登録し、監視を開始する
F-05	降車通知	ジオフェンス圏内に入ったとき音量最大化（Android）・プッシュ
F-06	ジオフェンス監視停止	登録済みのジオフェンスを全削除し、監視とバイブレーション
F-07	パーミッション要求	位置情報（常に許可）と通知のパーミッションを要求する
F-08	監視状態表示	現在の選択駅名と監視中かどうかの状態を画面に表示する
F-09	パーミッション状態表示	パーミッションが付与されているかどうかをアイコンの色で示す
F-10	状態の永続化	アプリを再起動しても監視中状態（目のアイコン）と選択駅の
F-11	駅名検索	駅名を入力して前方一致で検索し、ダイアログから路線を選ん
F-12	選択駅ジャンプ	選択中の駅が含まれる最初の路線の先頭へリストをスクロール
F-13	同名駅循環ジャンプ	同名駅が複数路線に存在する場合、ボタンを押すたびに次の路線

補足：優先度は以下の基準で分類。

- **必須:** これがないとアプリとして成り立たない
- **重要:** なくても動くが、ユーザー体験が大幅に低下する

---

4. 機能要件

このセクションの目的：各機能の﹁振る舞い﹂を自然言語で定義する。技術的な実装方法は書かず、﹁ユーザーから見てどう動くか﹂を記述する。

F-01 駅データ取得

```
概要:    アプリ起動時に外部APIから全路線・駅データを取得する
入力:    なし（自動取得）
処理:    外部APIサーバーへPOSTリクエストを送る
出力:    路線名・駅名・緯度経度のリスト
```

異常系:

- 通信エラー時はエラーメッセージを表示する
- データ取得中はローディング表示をする（ただし本実装では省略）

F-02 路線·駅一覧表示

```
概要:    取得した駅データを路線ごとにグループ化して一覧表示する
入力:    F-01で取得した駅データ
処理:    路線名をグループヘッダーとし、各駅を配下にリスト表示する
出力:    路線グループ化された駅リスト
```

表示項目（各駅）:

- 選択インジケーター（選択中は黄色の丸 / 非選択は白枠の丸）
- 駅名
- 緯度·経度

表示ルール:

- 路線名をヘッダーとして表示する
- データは取得順（API提供順）に表示する

F-03 駅選択

```
概要:    路線・駅一覧から降りたい駅を1つ選択する
入力:    駅アイテムのタップ
処理:    選択状態をアプリ内で保持する
出力:    選択駅名の画面表示更新、インジケーターの強調表示
```

動作ルール:

- 選択できる駅は1つのみ︵複数選択不可︶
- 別の駅をタップすると選択が切り替わる
- 選択した駅名は画面上部の︻現在の選択︼欄に表示される

F-04 ジオフェンス監視開始

```
概要:    選択した駅の座標にジオフェンスを登録し、監視を開始する
入力:    監視開始ボタン（目のアイコン）のタップ
処理:    選択駅の緯度経度を中心に、半径1000mのジオフェンスを登録する
出力:    監視中状態への遷移（ボタンが黄色に変わる）
```

動作ルール:

- ジオフェンスのトリガーは︻入場︵圏内への進入︶︼のみ
- 監視開始前に位置情報パーミッションが付与されていることを確認する
- パーミッションが未取得の場合は、パーミッション要求を促すメッセージを表示する

F-05 降車通知

```
概要:    ジオフェンス圏内に入ったとき、音量最大化・ループバイブレーション・プッシュ通知で知らせる
入力:    OSからのジオフェンス入場イベント（バックグラウンド動作）
処理:    音楽ストリームの音量を最大にし、通知を発火し、バイブレーションをユーザーが停止するまでループさせる
出力:    音量最大化（Android のみ・音楽ストリーム）
          プッシュ通知（タイトル「降りる駅アラーム」、ボディに駅名を表示）
          バイブレーション（ユーザーが停止操作するまで繰り返し）
```

動作ルール:

- アプリがバックグラウンドまたは終了状態でも動作する
- 音楽ストリームの音量を最大︵1.0︶に引き上げる︵Android のみ︶。システムUIの音量スライダーは表示しない
- 音量の復元は自動では行われない。ユーザーが手動で調整する
- バイブレーションは最大強度︵Android: intensity 255︶でパターンをループし続ける
- バイブレーションの停止はユーザーがアプリの停止ボタン︵×︶を押したときのみ行われる

- 通知タイトル: ﹁降りる駅アラーム﹂
- 通知本文: 対象駅名を含む

F-06 ジオフェンス監視停止

```
概要:    登録済みの全ジオフェンスを削除し、監視とバイブレーションを停止する
入力:    停止ボタン（×アイコン）のタップ
処理:    OSに登録済みのジオフェンスを全件削除し、ループ中のバイブレーションも即時停止する
出力:    非監視状態への遷移（目のアイコンが通常色に戻る）
```

動作ルール:

- 停止後は｀圏内に入っても通知は発火しない
- ループ中のバイブレーションは停止ボタンを押した瞬間に確実に止まる
- 選択駅の情報はリセットされる︵画面の選択表示が﹁(未選択)﹂に戻り｀SharedPreferencesからも削除される︶

F-07 パーミッション要求

```
概要:    ジオフェンスと通知の動作に必要なパーミッションをユーザーに要求する
入力:    パーミッション要求ボタン（錠前アイコン）のタップ
処理:    位置情報（Fine位置情報）→ 位置情報（常に許可）→ 通知の順に要求する
出力:    OSのパーミッションダイアログ表示、付与状況のアイコン更新
```

動作ルール:

- Fine位置情報 → 常に許可 → 通知の順に順次要求する
- 既に付与済みのパーミッションはスキップする
- ﹁常に許可﹂はOSの設定画面に誘導が必要な場合がある

F-08 監視状態表示

```
概要:    現在の選択駅名と監視中かどうかの状態を画面に表示する
表示内容:
  - 選択駅名（未選択時は「(未選択)」）
  - 監視中は「監視中」テキストを表示、停止中は非表示
```

F-09 パーミッション状態表示

```
概要:    パーミッションが付与済みかどうかをAppBarのアイコン色で示す
表示ルール:
  - パーミッション付与済み: 錠前アイコンが黄色
  - 未取得: 錠前アイコンが通常色（白）
  - 監視中: 目のアイコンが黄色
  - 停止中: 目のアイコンが通常色（白）
```

F-10 状態の永続化

```
概要:    アプリを再起動しても監視中状態（目のアイコン）と選択駅の両方を復元する
入力:    アプリ起動
処理:    起動時に SharedPreferences から監視状態・選択駅を読み込み、画面に反映する
出力:    前回終了時の監視状態と選択駅名がそのまま表示される
```

F-11 駅名検索

```
概要:    駅名を入力して前方一致で検索し、ダイアログから路線を選んで路線リストへジャンプする
入力:    テキストフォームへの駅名入力 + 検索ボタンタップ
処理:    入力された文字列で駅名を前方一致フィルタリングする
          ダイアログを表示し、該当駅と路線の一覧を表示する
          路線アイテムタップでダイアログを閉じ、その路線の先頭位置へリストをスクロールする
出力:    検索結果ダイアログ（駅名ヘッダー + 路線リスト）、リストのスクロール
```

表示ルール:

- 駅名は太字のヘッダーとして表示する
- 各ヘッダーの下に `その駅が属する路線名を ListTile で表示する
- 路線名をタップするとダイアログを閉じて `その路線の ExpansionTile 先頭へアニメーションスクロールする

異常系:

- 入力が空の場合: ¬︵未入力︶ ̄ と表示する
- 一致する駅が0件の場合: ¬該当する駅が見つかりませんでした ̄ と表示する

F-12 選択駅ジャンプ

```
概要:    現在選択中の駅が含まれる最初の路線へリストをスクロールジャンプする
入力:    「選択駅へジャンプ」ボタン（📍アイコン）のタップ
処理:    選択駅名を含む路線を先頭から探し、最初に見つかった路線の位置へジャンプする
出力:    リストが対象路線へアニメーションスクロールする
```

動作ルール:

- 駅が選択されていない場合はボタン自体が非表示︵透明ダミー表示︶になる
- ジャンプ後、同名駅の表示カウンターを 0︵先頭︶にリセットする

F-13 同名駅循環ジャンプ

```
概要:    同一駅名が複数路線に存在する場合、ボタンを押すたびに次の路線の同名駅へ順番にジャンプする
入力:    「次の同名駅へ」ボタン（↕アイコン）のタップ
処理:    選択駅名を含む全路線のインデックスを取得し、現在位置から次の路線へ循環ジャンプする
出力:    リストが次の同名駅を含む路線の位置へアニメーションスクロールする
```

動作ルール:

- 駅が選択されていない場合はボタン自体が非表示︵透明ダミー表示︶になる
- 路線が1つだけの場合は同じ路線に戻る︵循環︶
- ジャンプ順序は路線リストの並び順︵API取得順︶に従う

動作ルール:

タイミング	監視状態︵isSetStation︶	選択駅︵selectedStation︶
目のアイコンをタップ	`true` を保存	変更なし︵既に保存済み︶
駅をタップして選択	変更なし	選択駅の JSON を保存
停止ボタン︵×︶をタップ	削除	**削除**︵画面も ¬(未選択)⊢ に戻る︶
アプリ起動	読み込んで復元	読み込んで復元

- 注意: ジオフェンスの監視自体はOSが管理するため、アプリ再起動後も継続している。アイコンの状態だけがアプリ内で復元される

5. 画面一覧と画面遷移

> **このセクションの目的**：アプリにどんな画面があり、ユーザーがどうやって画面を移動するかを定義する。

5.1 画面一覧

画面ID	画面名	種別	概要
S-01	ホーム画面	画面	アプリの唯一のメイン画面。駅名検索・路線・駅一覧・選
S-02	検索結果ダイアログ	ダイアログ	駅名検索の結果を表示する。路線タップで路線リスト

> **補足**：本アプリは基本的に単一画面構成。検索結果ダイアログのみ AlertDialog で表示し、それ以外はホーム画面1枚で完結させるシンプルな設計を採用している。

5.2 画面レイアウト図

```
┌──────────────────────────────────────────────┐
│                AppBar                         │
│  降りタイマー              [🔒][  ][×]        │
│                            (錠前)(目)(停止)     │
├──────────────────────────────────────────────┤
│                                               │
│  [  駅名を検索...                [×]  ] [検索]  │  ← 検索フォーム（F-11）
│                                               │
│  選択中: 渋谷駅  [📍] [↕]                     │  ← [📍]=F-12 [↕]=F-13
│                                               │
│                                               │
├──────────────────────────────────────────────┤
│  ┌─ 山手線 ──────────────────────────────┐    │
│  │ ○  東京駅    35.6812, 139.7671         │    │
│  │ ○  上野駅    35.7138, 139.7774         │    │
│  │ ●  渋谷駅    35.6580, 139.7016  ← 選択 │    │
│  │ ○  新宿駅    35.6896, 139.7006         │    │
│  └──────────────────────────────────────┘    │
│  ┌─ 中央線 ──────────────────────────────┐    │
│  │ ○  東京駅    35.6812, 139.7671         │    │
│  │ ○  新宿駅    35.6896, 139.7006         │    │
│  │ ...                                    │    │
│  └──────────────────────────────────────┘    │
└──────────────────────────────────────────────┘
```

【検索結果ダイアログ（S-02)】

```
┌──────────────────────────────┐
│  "渋谷" の検索結果              │
│ ──────────────────────────────│
│  渋谷駅                        │  ← 駅名（太字ヘッダー）
│    🚃 山手線                   │  ← タップで路線へジャンプ
│    🚃 東急東横線               │
│  渋谷本町                      │
│    🚃 都営大江戸線             │
└──────────────────────────────┘
```

5.3 AppBarボタンの操作詳細

ボタン	アイコン	動作	状態
パーミッション要求	🔒︵錠前︶	位置情報·通知パーミッションを順次要求する	付与済み=黄色
監視開始	︵目︶	選択駅のジオフェンス監視を開始する	監視中=黄色
監視停止	×︵×︶	全ジオフェンスを削除して監視を停止する	常に白

5.4 操作フロー

```
                    アプリ起動
                        │
                        ▼
            ┌──────────────────┐
            │     ホーム画面     │
            │                  │
            │  [検索フォーム]    │
            │  [路線・駅一覧]    │
            │  [現在の選択]      │
            └──────────────────┘
             │     │    │    │    │
    ┌────────┘     │    │    │    └────────────────┐
    │              │    │    │                     │
    │ 検索ボタン    │    │    │ 🔒タップ            │    タップ
    ▼              │    │    ▼                     ▼
検索結果           │    │ パーミッション        監視開始
ダイアログ         │    │ 要求ダイアログ        (ジオフェンス登録)
    │              │    │  (OS標準UI)
    │ 路線タップ   │    │
    ▼              │    │
路線へジャンプ     │    │
(スクロール)       │    │
                   │    │
            駅をタップ │
                   ▼    │
                 駅選択  │
              (状態更新)  │
                        │
                       ×タップ
                        │
                        ▼
                     監視停止
                 (ジオフェンス削除)
```

6. データ設計

> **このセクションの目的**：アプリが扱う「データ」の種類と関係を定義する。詳細設計書ではフィールドの型まで書くが、基本設計書では「どんなデータがあるか」「データ同士がどう関係するか」を概念レベルで記述する。

6.1 データの種類

データ名	概要	主な項目
路線情報	東京の鉄道路線の基本情報	路線番号、路線名、所属駅リスト
駅情報	各駅の詳細情報	駅ID、駅名、住所、緯度、経度
アプリ状態	アプリの現在の動作状態	ジオフェンス監視中かどうか、選択駅情報（どちらも永

6.2 データの関係

```
┌──────────────────┐        ┌──────────────────┐
│    路線情報       │        │    駅情報         │
│                  │ 1   N  │                  │
│  - 路線番号       │────────│  - 駅ID           │
│  - 路線名        │        │  - 駅名          │
│                  │        │  - 住所           │
│                  │        │  - 緯度           │
│                  │        │  - 経度           │
└──────────────────┘        └──────────────────┘
                                     │
                                     │ ユーザーが1つ選択
                                     ▼
                            ┌──────────────────┐
                            │    選択駅         │
                            │ (アプリ状態)      │
                            │  - 選択中の駅情報  │
                            │  - 監視中フラグ    │
                            └──────────────────┘
```

関係の説明:

- 1つの路線に対して、N個の駅がある（1対多の関係）
- ユーザーが選択できる駅は任意の1駅のみ

6.3 データの取得タイミング

データ	取得タイミング	保持方法
路線・駅情報	アプリ起動時に全件取得（keepAlive）	Riverpod Providerでキャッシュ
選択駅情報	ユーザーの駅タップ時に更新	Riverpod Notifierで管理
監視状態	ジオフェンス開始・停止操作時に更新	Riverpod Notifier + SharedPreferences（永続化）
選択駅情報	駅タップ時に保存、停止ボタンで削除	SharedPreferences（JSON文字列で永続化）

> **補足**：`SharedPreferences` とは、アプリを終了しても消えない小さなデータを端末に保存する仕組み。Androidでは `XML` ファイル、iOSでは `NSUserDefaults` が裏側で使われる。監視状態の `true/false` のような小さなフラグの保存に適している。

> **補足**：「keepAlive」とは一度取得したデータをアプリのライフサイクル全体を通して保持し続ける設定。駅データは頻繁に変わらないため、再取得せずに使い回す。

---

7. 外部インターフェース設計

> **このセクションの目的**：アプリが外部（サーバー）とやりとりする「窓口」を定義する。基本設計書ではエンドポイントの一覧と概要を記述し、詳細なリクエスト/レスポンスのフォーマットは詳細設計書に譲る。

7.1 API一覧

API-ID	メソッド	エンドポイント	概要	対応機能
API-01	POST	/BrainLog/api/getTokyoTrainStation	東京の全路線・駅情報を取得	F-01

7.2 通信シーケンス

> **補足**：シーケンス図とは、「誰が」「いつ」「何を」やりとりするかを時系列で示す図。

アプリ起動時:

```
Flutterアプリ              外部APIサーバー
    │                          │
    │  POST /BrainLog/api/      │
    │  getTokyoTrainStation     │
    │──────────────────────────►│
    │                          │
    │  JSON（路線・駅データ）     │
    │◄──────────────────────────│
    │                          │
    │ 路線・駅一覧を画面に表示    │
```

駅選択～監視開始:

```
ユーザー          Flutterアプリ          OSジオフェンスサービス
   │                    │                          │
   │ 駅をタップ         │                          │
   │──────────────────→│                          │
   │                    │ 選択状態を更新            │
   │  画面更新          │                          │
   │←──────────────────│                          │
   │                    │                          │
   │   ボタンをタップ   │                          │
   │──────────────────→│                          │
   │                    │ ジオフェンスを登録        │
   │                    │──────────────────────────→│
   │                    │ 登録完了                  │
   │                    │←──────────────────────────│
   │  監視中表示        │                          │
   │←──────────────────│                          │
   │                    │                          │
   │  (駅の500m圏内に入る)                          │
   │                    │←──────────────────────────│
   │                    │ コールバック発火          │
   │                    │                          │
   │  プッシュ通知      │                          │
   │←──────────────────│                          │
```

---

8. 非機能要件

> **このセクションの目的**：「動くこと」以外の品質要件を定義する。機能要件が「何ができるか」なら、非機能要件は「どれくらいの品質で動くか」を定義するもの。

8.1 性能要件

項目	要件	備考
初期表示	3秒以内に路線・駅一覧を表示	ネットワーク環境に依存
ジオフェンス登録	タップから2秒以内に登録完了	ネイティブAPIの応答に依存

項目	要件	備考
通知発火	圏内進入後、できる限り短時間で発火	OSや省電力設定に依存するため厳密な

8.2 可用性要件

項目	要件
バックグラウンド動作	アプリが終了状態でもジオフェンス監視・通知が機能すること
サーバー停止時	エラーメッセージを表示。アプリはクラッシュしないこと
パーミッション未取得時	適切なガイダンスを表示し、アプリが停止しないこと

8.3 ユーザビリティ要件

項目	要件
操作性	駅選択から監視開始まで3タップ以内で完了できること
状態確認	監視状態・選択駅が画面一目で確認できること
パーミッション	パーミッション未取得時に、分かりやすいガイダンスを表示すること
テーマ	ダークテーマを採用する
バイブレーション	到着時のバイブレーションはユーザーが停止操作するまでループし続けること
状態復元	アプリ再起動後も監視状態（目のアイコン）が前回の状態を維持していること

8.4 保守性要件

項目	要件
API接続先	1箇所の設定（`path.dart`）で変更可能
データモデル	freezedによるコード生成で型安全性を担保
状態管理	Riverpodで統一。Widgetから直接API通信しない
ファイル構成	役割別にディレクトリを分割（model / controllers / screens）

8.5 セキュリティ要件

項目	現状	将来的な対応案
通信暗号化	HTTP︵非暗号化︶	HTTPS化を推奨
位置情報の扱い	通知発火のみに使用。外部には送信しない	変更なし
認証	なし	必要に応じてAPI Key等を導入

---

9. 用語集

> **このセクションの目的：**ドメイン特有の用語や、プロジェクト内での独自の言葉遣いを定義する。

用語	意味
ジオフェンス	地図上に設定した仮想的な境界線。境界線の通過時にイベントが発火する
圏内進入	ジオフェンスで設定した円の内側にデバイスが入ること︵エントリーイベン
バックグラウンド動作	アプリが画面に表示されていない･終了している状態でも機能する動作
パーミッション	アプリがOS機能︵位置情報･通知︶を使うためのユーザー許可
常に許可	位置情報パーミッションの一種。アプリがバックグラウンドでも位置情報を
ジオフェンスサービス	iOSのCoreLocation、AndroidのGeofencing APIなど、OSが提供するジオフ
keepAlive	Riverpod Providerの設定。アプリのライフサイクル全体でデータを保持し続
Provider	Riverpodの仕組み。データの取得･保持･配信を担当する部品
freezed	Dartのコード生成ライブラリ。イミュータブル︵変更不可︶なモデルクラス

改訂履歴

版	日付	
1.0	2026-02-28	初版作成
1.1	2026-03-01	ジオフェンス半径を500m→1000mに変更（1.2・2.4・F-04）。バイブレーションの
1.2	2026-03-03	選択駅の永続化をF-10・6.1・6.3に追記。停止時に選択駅も消去される仕様をF-06
1.3	2026-03-03	ジオフェンス侵入時の音量最大化（Android）をF-05機能一覧・動作ルールに追記
1.4	2026-03-07	駅名検索機能を追加（F-11新規追加・画面一覧にS-02追加・5.2レイアウト図更新・
1.5	2026-03-08	選択駅ジャンプ機能を追加（F-12新規追加）。同名駅循環ジャンプ機能を追加（